

课程笔记

[LLM Agents F25] LLM Agents Overview 与训练路线

基于 Yann Dubois 授课内容整理

2026-04-02



视频作者/频道: Berkeley RDI

发布日期: 2025-09-29

视频时长: 1:58:21

视频链接: <https://www.youtube.com/watch?v=r1qZpYAmqmg>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 课程定位：这门课在 Agent 版图中的角色	3
1.1 讲者背景与课程目标	3
1.2 从“聊天模型”到“可执行主体”	3
1.3 关键画面：课程开场声明	4
1.4 本章小结	4
2 训练流水线总览：从 SFT 到 Reasoning RL 再到 Agent RL	4
2.1 三阶段训练架构	4
2.2 Reasoning RL 的定位与输入输出	5
2.3 关键画面：Reasoning RL 段落	5
2.4 本章小结	5
3 数据与奖励：训练质量的第一约束	6
3.1 数据结构从“样本”变为“轨迹”	6
3.2 奖励设计：局部奖励与终局奖励的平衡	6
3.3 数据质量与评估质量的耦合	6
3.4 本章小结	6
4 算力与成本：从训练可行到产品可用	7
4.1 课程中的成本讨论框架	7
4.2 成本分解模型	7
4.3 本章小结	8
5 工具使用训练：从语义能力到操作能力	8
5.1 为什么工具使用是 Agent 训练的分水岭	8
5.2 合成工具数据与三模型模拟	8
5.3 合成数据的边界	9
5.4 本章小结	9
6 评估体系：为什么“evaluation is the key”	10
6.1 闭集评估与开集评估	10
6.2 闭集评估的优缺点	10
6.3 开集评估的核心困难	10
6.4 评估设计模板	11
6.5 本章小结	11
7 长时 Rollout 与系统工程瓶颈	11
7.1 为什么 Agent 训练很快变成系统问题	11
7.2 内存层级与通信	11
7.3 面向 Agent 的系统优化策略	12
7.4 本章小结	12

8 安全、鲁棒性与上线策略	13
8.1 从训练安全到运行时安全	13
8.2 鲁棒性评估应覆盖失败恢复	13
8.3 分阶段上线	13
8.4 本章小结	13
9 组织与研发流程：把课程观点转成执行机制	14
9.1 跨职能协同的必要性	14
9.2 实验管理与复盘模板	14
9.3 本章小结	14
10 实战推演：三类 Agent 项目的训练与交付路径	14
10.1 路径 A：内部知识问答型 Agent	14
10.2 路径 B：流程自动化型 Agent	15
10.3 路径 C：研究助理型 Agent	15
10.4 三条路径的共性工程清单	15
10.5 本章小结	16
11 评估样例库与排障手册：把质量控制写成文档	16
11.1 评估样例库的分层构造	16
11.2 排障优先级：先判定是评估问题还是策略问题	16
11.3 标准化排障流程	17
11.4 长表：常见故障与处理策略	17
11.5 验收前检查清单	18
11.6 本章小结	18
12 总结与延伸	18
12.1 全课总结表	18
12.2 关键结论回顾	19
12.3 进一步阅读	19

1 课程定位：这门课在 Agent 版图中的角色

1.1 讲者背景与课程目标

本讲由 Yann Dubois 主讲，定位是整个 Agentic AI MOOC 中关于“训练 LLM 成为 Agent”的总览课。课程不是单一算法讲解，而是把训练、评估、系统、成本、部署连成一个统一工程视角。讲者在开场强调，这是一版补录课程，核心目的是把原课堂因技术问题中断的内容完整补齐。

核心命题：Agent 能力不是单点突破，而是系统协同

如果只看模型参数量、只看某个 benchmark 排名，常常会误判进展。课程不断重复一个判断：决定上限的是数据质量、评估闭环、系统吞吐、推理策略与成本约束的耦合效果，而不是任何单个模块。

1.2 从“聊天模型”到“可执行主体”

课程把 Agent 定义为能在环境中持续交互、调用工具、跨步骤保持目标一致性的模型系统。与传统单轮问答相比，Agent 有三个结构性变化：

1. 输出不再是最终文本，而是中间动作序列（tool call / code / API）。
2. 成功标准不再是“看起来对”，而是是否在外部环境中完成目标。
3. 推理成本由“单次回答”转为“长时 rollout 的累计资源”。

课程的组织逻辑

从内容编排看，讲者先定义训练流水线，再讨论为什么“evaluation is the key”，随后进入成本与系统瓶颈，最后回到 tool use 和 open-ended evaluation 的落地挑战。这种顺序对应真实研发节奏：先把环路搭起来，再优化每个瓶颈。

1.3 关键画面：课程开场声明

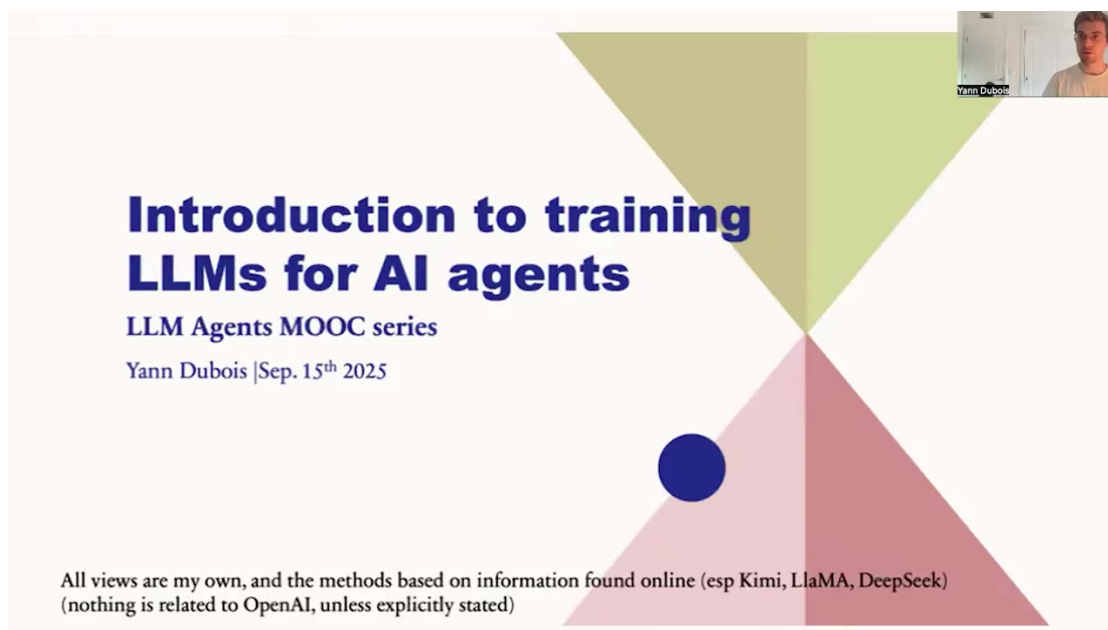


图 1: 讲者在开场说明补录背景与课程目标（约 00:00:42）

1.4 本章小结

本章建立了课程的基本坐标：这不是“某个技巧”的集合，而是 Agent 训练的端到端工程方法。后续所有技术细节都围绕一个问题展开，即如何把模型能力稳定转化为可验证、可扩展、可部署的 Agent 行为。

2 训练流水线总览：从 SFT 到 Reasoning RL 再到 Agent RL

2.1 三阶段训练架构

课程中将主干流程抽象为三层：监督微调（SFT）奠定基础行为，推理强化学习（Reasoning RL）增强中间思维质量，Agent 强化学习（Agent RL）把能力迁移到环境交互。

阶段	目标	典型风险
SFT	学习指令遵循、格式稳定、基础任务完成率	过拟合模板，推理深度不足
Reasoning RL	提升多步推导、策略反思、错误恢复能力	奖励设计失真，出现 reward hacking
Agent RL	在真实或模拟环境中优化动作序列与终局成功率	rollout 成本高，评估噪声大，训练不稳定

表 1: 课程中的三阶段训练框架

为什么 Reasoning RL 要单独成段

讲者明确指出，推理强化学习不是“顺便做一下”的附属步骤，而是连接文本能力与 Agent 行为的关键桥梁。没有稳定的中间推理，Agent 在工具链中的错误会沿时间轴放大，最终在长任务中崩溃。

2.2 Reasoning RL 的定位与输入输出

在课程语境下，Reasoning RL 的训练对象不是单一答案，而是“思考过程 + 决策路径”。这要求训练样本包含可对齐的中间轨迹，奖励函数也要能区分“虽然答对但过程不可复用”与“过程稳定可迁移”的区别。

常见误区：把推理 token 量当作推理质量

更长的 chain-of-thought 不等于更好的策略。课程强调真正的收益来自 **有结构的推理行为**，例如何时回退、何时重计划、何时调用工具，而不是单纯延长输出长度。

2.3 关键画面：Reasoning RL 段落

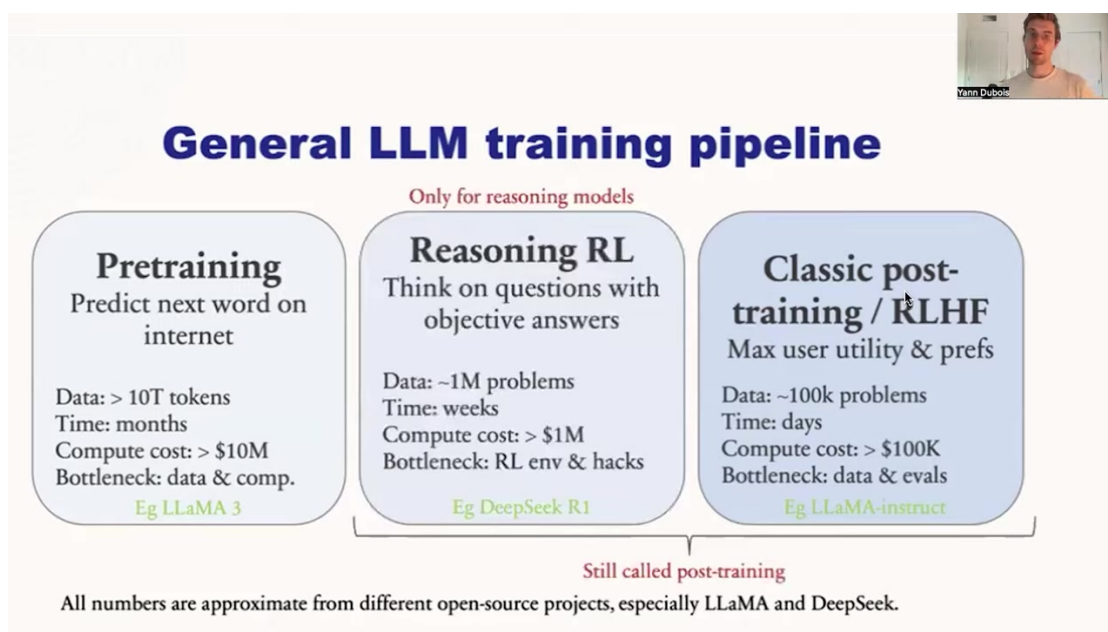


图 2: 课程在早期明确提出“reasoning reinforcement learning” (约 00:05:10)

2.4 本章小结

SFT、Reasoning RL、Agent RL 构成了一个由浅入深的能力迁移链。Reasoning RL 的存在不是可选项，而是把“会说”变成“会做”的中介层。

3 数据与奖励：训练质量的第一约束

3.1 数据结构从“样本”变为“轨迹”

Agent 训练中的数据不只是问答对，而是环境状态、动作、工具调用结果、延迟反馈与最终 outcome 的组合轨迹。课程反复提醒，如果数据只保留最终答案，模型会失去“如何到达答案”的学习信号。

轨迹数据的四个必要字段

1. 上下文状态：任务背景、可用工具、历史动作。
2. 决策动作：模型当前选择的操作与参数。
3. 环境反馈：工具返回、执行日志、外部错误信息。
4. 终局标签：成功/失败、质量评分、代价统计。

3.2 奖励设计：局部奖励与终局奖励的平衡

课程给出的实践倾向是，尽量让奖励与最终任务完成度一致，同时为关键中间步骤设置轻量 shaping。若中间奖励过强，模型会为了拿分而偏离真实任务；若只有终局奖励，训练又会因为稀疏信号而效率低下。

奖励设计的工程判据

有效奖励通常满足三点：可计算、可解释、可扩展。可计算意味着能在流水线上自动生成；可解释意味着失败样本能回溯问题来源；可扩展意味着在新任务上不需要全部重写。

3.3 数据质量与评估质量的耦合

讲者在早段就强调“data, evaluation and systems”同等重要。原因是坏评估会反向污染数据筛选；坏数据又会让评估指标失真，形成负反馈。研发上必须把数据治理和评估治理一起做，而不是分部门串行。

误区：先把模型训大再补评估

在 Agent 场景里，这种顺序通常成本更高。模型越大、rollout 越长，后补评估意味着要在更高单次实验成本下重新定位问题，迭代速度会显著下降。

3.4 本章小结

Agent 训练阶段的数据单位是轨迹，不是答案；奖励函数要在稀疏终局目标和可学习中间信号间做平衡。最关键的是，数据与评估必须并行设计。

4 算力与成本：从训练可行到产品可用

4.1 课程中的成本讨论框架

讲者在多个位置给出“compute cost around ...”的数量级提示，核心目的是让研究决策与资源现实对齐。即便某条路径在离线实验有效，也必须评估其训练时间、并行开销、推理成本和部署成本。

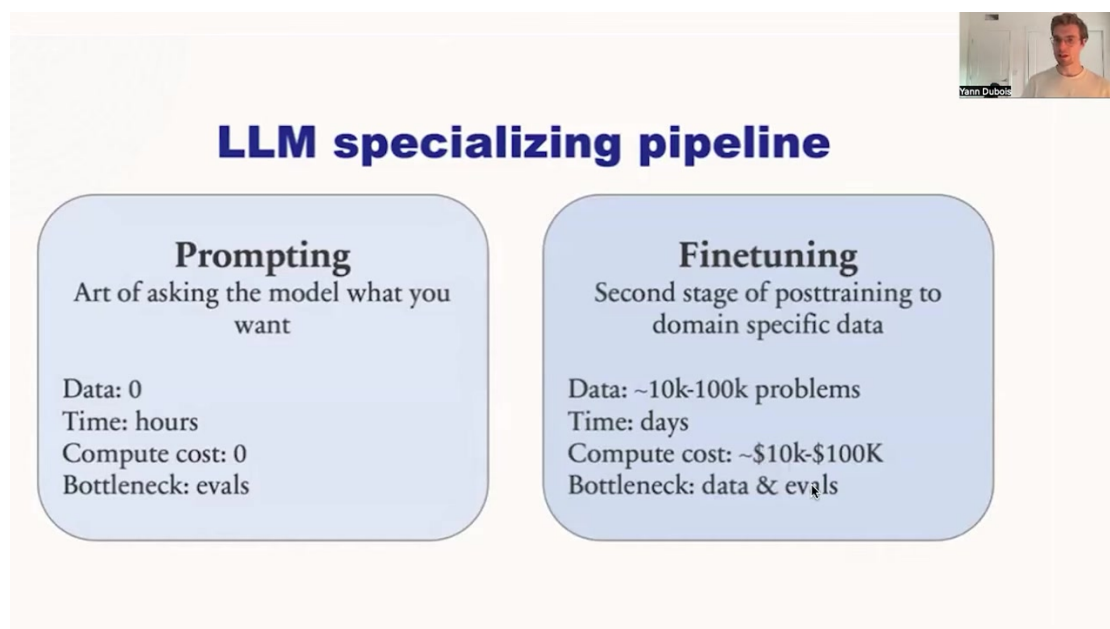


图 3: 课程中以数量级方式讨论训练计算成本（约 00:12:05）

训练成本与推理成本必须分开记账

课程明确指出，很多实验只优化训练资源，却忽略 inference cost。对于 Agent 产品，推理侧往往是持续性支出，且会随着任务链路拉长而成倍增长，因此不能只看训练阶段的“一次性”成本。

4.2 成本分解模型

可把总成本粗略拆为：

$$C_{\text{total}} = C_{\text{data}} + C_{\text{train}} + C_{\text{eval}} + C_{\text{inference}} + C_{\text{ops}}$$

其中最容易被低估的是 C_{eval} 与 $C_{\text{inference}}$ 。Agent 场景下，评估本身可能需要复杂环境复现，推理则包含多轮调用、工具耗时和失败重试。

对齐研发与产品的成本仪表盘

建议至少监控以下指标：

- 每次有效策略改进的平均 token 成本；
- 每个成功任务的平均 rollout 长度；
- 失败重试率与重试引入的额外推理费用；
- 评估覆盖率与评估单次执行成本。

4.3 本章小结

成本不是财务层面的后置问题，而是训练策略选择的一阶约束。忽略推理和评估成本，会让“研究上有效”的方案无法产品化。

5 工具使用训练：从语义能力到操作能力

5.1 为什么工具使用是 Agent 训练的分水岭

课程在后半段详细讨论 tool use，原因在于工具调用让模型进入“动作有副作用”的世界。模型不仅要决定“说什么”，还要决定“做什么”，并承担执行后果。

工具调用能力的本质

工具调用不是函数格式学习，而是决策学习。需要同时解决何时调用、调用哪个工具、参数如何构造、失败后如何恢复四类问题。

5.2 合成工具数据与三模型模拟

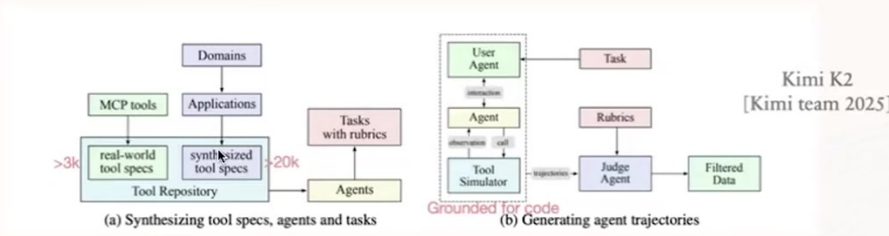
讲者介绍了通过 synthetic tools 扩展训练空间的做法：用一个代理模型扮演用户、一个模型扮演工具环境、一个模型作为待训练 agent，在闭环中生成大量多样交互轨迹。



Yann Dubois

SFT pipelines can be complex: eg Kimi k2

- SFT pipelines can use complex system to build useful SFT data
- Eg Kimi K2 uses LLM simulated user & tools, and rubric based rejection sampling to build data for agentic tool use



Kimi K2
[Kimi team 2025]

图 4: 课程讨论 synthetic tools 与 tool-simulating LM (约 01:06:01)

三角色模拟的收益

- 扩展工具覆盖面，降低真实工具接入成本；
- 更快制造 corner case，提升鲁棒性；
- 在可控环境中复现实验，便于 ablation。

5.3 合成数据的边界

合成数据虽高效，但容易产生分布偏移。若工具模拟器过于理想化，模型会学到“仿真世界最优策略”，而非真实 API 世界的稳健策略。

合成工具训练的三类偏差

1. 错误类型偏差：仿真器未覆盖真实系统的脏数据与异常码。
2. 延迟偏差：真实网络波动和队列抖动在仿真中被简化。
3. 权限偏差：真实鉴权、配额、审计流程被忽略。

5.4 本章小结

工具使用训练决定 Agent 能否在真实环境落地。合成工具是加速器，但必须与真实工具回放结合，否则泛化能力会被高估。

6 评估体系：为什么 “evaluation is the key”

6.1 闭集评估与开集评估

课程将评估分为 close-ended 与 open-ended。前者可以通过标准答案或结构化判定快速打分，后者通常没有唯一答案，需要多维标准与更复杂评审机制。

原话级观点

讲者在约 01:23:20 强调：“*evaluation is the key*”。这不是修辞，而是工程事实。没有高质量评估，训练改进无法可靠排序，资源投入会快速失效。

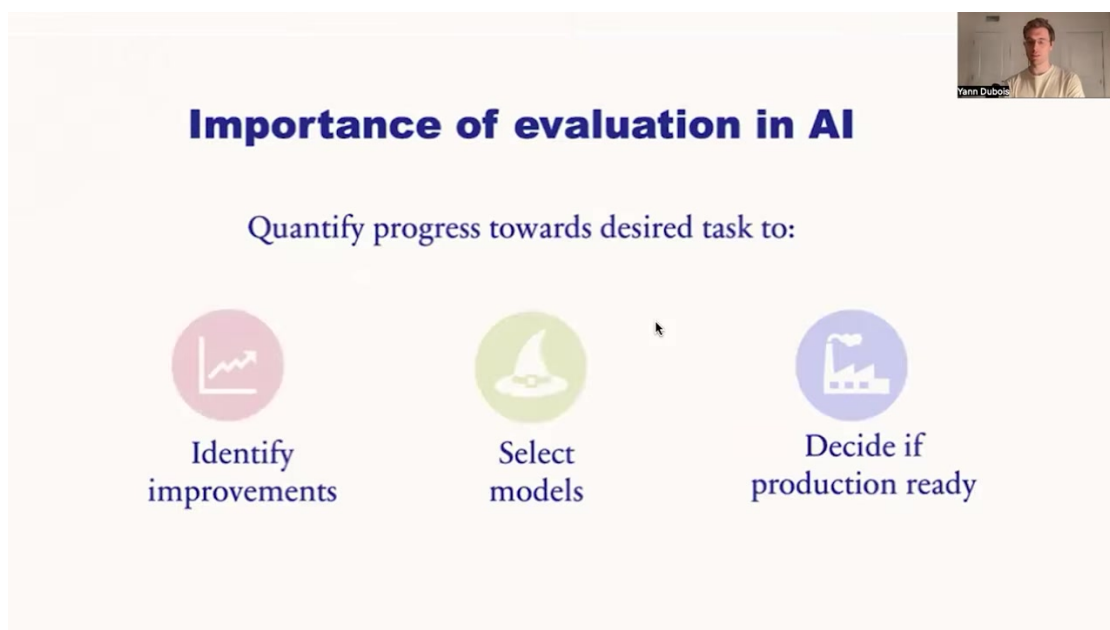


图 5: 课程明确指出评估是最关键环节（约 01:23:20）

6.2 闭集评估的优缺点

闭集评估的优势是低成本、高自动化、易比较；缺点是可能被 prompt 和打分器偏差“刷分”。在 Agent 场景，闭集评估适合做快速回归，但不能单独作为上线依据。

闭集评估的推荐用途

- 每日训练回归与 smoke test；
- 新策略的快速筛选；
- 失败类型的早期预警。

6.3 开集评估的核心困难

开集任务（如复杂客服、多轮协作、开放式写作）通常存在多解、长路径和偏好冲突。课程提醒，单一准确率指标不再有效，必须引入过程指标与结果指标共同约束。

开集评估中最常见的两个坑

1. 用“像人类”取代“完成目标”：观感好但任务失败。
2. 用单次胜负取代稳定性：一次成功掩盖高方差问题。

6.4 评估设计模板

层级	指标	示例
结果层	任务完成率、成功成本	Ticket 关闭率、每单 token 花费
过程层	错误恢复率、重计划次数	API 失败后是否能自动纠错
安全层	越权率、敏感操作拦截率	是否触发越权写入或危险调用
体验层	延迟、可解释性评分	用户等待时间、行动理由可读性

表 2: 课程观点落地为可执行评估模板

6.5 本章小结

评估决定训练方向与资源分配，是 Agent 研发的“方向盘”。闭集评估负责速度，开集评估负责真实性，两者缺一不可。

7 长时 Rollout 与系统工程瓶颈

7.1 为什么 Agent 训练很快变成系统问题

课程指出，随着任务长度增长，训练瓶颈常从“算子吞吐”转向“异步调度、内存管理、通信开销”。Agentic rollouts 越长，等待 I/O 和环境反馈的时间占比越高，GPU 可能并未被充分利用。

长时 rollout 的资源形态

- 计算资源：策略前向与反向传播；
- 存储资源：轨迹缓存、日志、回放数据；
- 通信资源：多机同步、参数广播、环境 RPC；
- 调度资源：异步任务池、失败重试、超时控制。

7.2 内存层级与通信

课程后段对 GPU memory hierarchy 做了专门解释，强调“算得快”并不总是瓶颈，“拿数据慢”往往才是。共享内存、缓存、全局显存、跨卡通信都直接影响 Agent 训练效率。



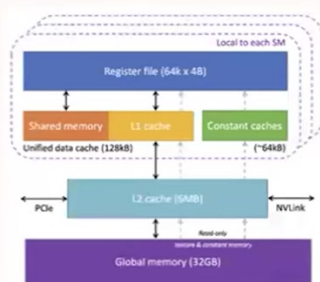
Yann Dubois

Systems 101: GPUs

- Massively parallel
- Fast matrix multiplication
- Compute > memory & communication
- Memory hierarchy:
 - Closer to cores $\Rightarrow$ faster but less memory
 - Further from cores $\Rightarrow$ more memory but slower

TABLE IV
THE MEMORY ACCESS LATENCIES

Memory type	CPI (cycles)
Global memory	290
L2 cache	200
L1 cache	33
Shared Memory (ld/st)	(23/19)



The diagram illustrates the GPU memory hierarchy. At the top, a 'Register file (64k x 48)' is shown. Below it, a dashed box labeled 'Local to each SM' contains 'Shared memory', 'L1 cache', and 'Constant caches'. The 'Unified data cache (128KB)' is shown below the L1 cache. The 'L2 cache (5MB)' is shown below the unified data cache. The 'Global memory (32GB)' is shown at the bottom. Arrows indicate data flow between these components and external interfaces like PCIe and NVLink.

图 6: 课程讨论内存层级与通信限制 (约 01:35:00)

工程结论

当 rollout 进入长链路后，系统优化优先级往往应从“再堆算力”改为“降通信、降等待、降无效重试”。否则训练账单增加，但有效学习信号并未同步增长。

7.3 面向 Agent 的系统优化策略

1. 异步 rollout 与训练解耦，减少互相阻塞；
2. 把高频短工具调用本地化缓存，降低 RPC 往返；
3. 对轨迹做分级存储，热数据放快存，冷数据批量归档；
4. 用失败类型路由替代统一重试，减少无效计算。

系统优化中的错配风险

若评估口径没同步更新，系统优化可能带来“看似更快但更差”的假象。例如吞吐提高了，但平均任务质量下降；或延迟降低了，但错误恢复率明显下滑。

7.4 本章小结

长任务 Agent 的上限高度依赖系统工程能力。训练与系统是一体两面，不做系统优化会让算法增益被基础设施损耗掉。

8 安全、鲁棒性与上线策略

8.1 从训练安全到运行时安全

课程虽以训练为主，但其评估框架天然延伸到上线安全。Agent 的风险点包括越权工具调用、错误自动化放大、长链路偏航和不可解释决策。

安全策略分层

1. 训练层：加入反例轨迹和拒绝策略数据；
2. 推理层：策略约束、动作白名单、敏感操作确认；
3. 监控层：实时审计、异常回滚、人类接管阈值。

8.2 鲁棒性评估应覆盖失败恢复

多数 benchmark 聚焦最终结果，但生产环境更关心“失败后是否能自救”。课程关于 open-ended evaluation 的讨论可直接映射到该问题：需要在评估中显式纳入恢复路径质量。

鲁棒性观测指标

- 首次失败后的恢复成功率；
- 恢复所需额外步骤数；
- 恢复过程中新增风险动作比例；
- 恢复后结果是否满足业务阈值。

8.3 分阶段上线

对 Agent 产品，课程思想对应的上线方法是“先可控，再扩域”：先在闭集任务上线高置信动作，再逐步引入开集任务，并持续校准评估器。

上线反模式

一次性放开所有工具权限、把开集任务直接按闭集规则打分、缺少失败复盘机制，这三种做法在 Agent 场景会显著放大风险与成本。

8.4 本章小结

Agent 安全不是额外模块，而是训练、评估、推理、运维的联合产物。鲁棒性要通过失败恢复能力来度量，而非只看一次成功率。

9 组织与研发流程：把课程观点转成执行机制

9.1 跨职能协同的必要性

根据课程对“data-eval-systems”的强调，Agent 团队不能按传统“模型组/平台组/应用组”完全割裂。至少在关键迭代周期内，数据、评估、系统、产品应共用一套优先级与实验台账。

最小可执行组织单元

建议建立“训练-评估-系统”三人小组：一人负责策略实验，一人负责评估设计，一人负责基础设施瓶颈优化。每周以统一任务集复盘，而不是各自汇报局部指标。

9.2 实验管理与复盘模板

字段	记录内容	目的
假设	本次改动要解决的具体问题	防止“试试也行”的随机实验
评估口径	闭集 + 开集指标、样本集版本	保证实验可比较
成本账单	训练/评估/推理分项资源消耗	评估 ROI，避免隐性亏损
失败复盘	失败类型与恢复路径分析	沉淀下一轮数据与规则

表 3: 将课程理念落地为团队实验管理模板

一句话流程准则

先定义成功，再开始训练；先定义失败恢复，再扩大权限。

9.3 本章小结

课程的价值不仅是技术路线，更是研发方法。将数据、评估、系统协同固化为流程，才能持续获得可复现增益。

10 实战推演：三类 Agent 项目的训练与交付路径

10.1 路径 A：内部知识问答型 Agent

这是最常见也是最容易被低估难度的一类。表面上像传统 RAG 问答，实际上只要涉及多轮澄清、跨系统检索与动作执行，就已经进入 Agent 设计范畴。基于本课方法，可采用“轻量 SFT + 任务化闭集评估 + 受限工具调用”的策略启动。

路径 A 的落地节奏

1. 第 1 周建立任务分解与工具白名单，只允许只读操作。
2. 第 2 周补齐 close-ended evaluation，覆盖高频 FAQ 与关键业务术语。
3. 第 3 周引入有限 open-ended 任务，跟踪失败恢复率与越权率。
4. 第 4 周再扩充上下文窗口和工具集合，避免早期过度复杂化。

这种路径的成功关键不是追求最强模型，而是快速建立“可量化反馈”。若没有评估闭环，团队很容易在提示词和模板工程里反复迭代，却无法证明任务完成率是否真正改善。

10.2 路径 B：流程自动化型 Agent

此类项目通常连接工单、审批、CRM、BI 等业务系统，目标是减少人工流程跳转。它对工具调用准确性和异常恢复能力要求高，训练策略应更偏向 Agent RL 与策略约束。

路径 B 的主要风险

流程型 Agent 最怕“看似完成，实则破坏状态一致性”。例如自动填单成功率高，但字段语义错配；自动发起流程成功率高，但审批链漏节点。这类问题在仅看闭集准确率时很难暴露。

阶段	目标	验证重点
设计期	定义动作空间与权限边界	是否存在危险动作未隔离
训练期	提升多步流程完成率	失败后是否能回滚并重计划
试运行	降低人工介入频次	自动化收益是否覆盖推理成本
扩张期	横向接入更多流程	新流程接入后旧流程是否退化

表 4: 流程自动化型 Agent 的分阶段交付要点

10.3 路径 C：研究助理型 Agent

研究助理型 Agent 需要检索文献、生成实验计划、执行脚本、汇总报告，天然 open-ended 且长时链路。课程中的“evaluation is the key”在这类场景体现最明显：没有多维评估，团队会很快被“生成内容看起来不错”的假象误导。

研究助理 Agent 的幻觉风险更隐蔽

这类系统经常出现“论证结构完整但证据链断裂”的输出。若评估只看语言流畅度或主观可读性，模型会被激励去写更像论文的文本，而非更真实的研究结论。

10.4 三条路径的共性工程清单

1. 必须先定义任务完成的机器可判定标准，再设计训练数据。

2. 必须在评估中记录失败恢复路径，而不仅是最终成败。
3. 必须把推理成本和人工复核成本一起纳入 ROI。
4. 必须有最小权限原则和紧急回滚通道。
5. 必须做版本化评估集，避免指标漂移导致误判。

课程观点在实战中的映射

不论是哪一类 Agent 项目，真正的交付单位都不再是“模型版本”，而是“模型 + 评估 + 系统 + 运行策略”的组合版本。只有组合版本才具有可复现业务价值。

10.5 本章小结

三类项目虽然目标不同，但工程规律一致：先评估、再训练；先约束、再扩权；先可复现、再追求规模。课程中的方法论可以直接作为项目启动模板。

11 评估样例库与排障手册：把质量控制写成文档

11.1 评估样例库的分层构造

为了避免“新策略提升 A 指标却破坏 B 能力”，建议将评估集划分为核心样例库、对抗样例库和回归样例库三层。核心库负责覆盖主业务，对抗库负责暴露脆弱点，回归库负责监控历史问题是否复发。

建议的样例库配比

- 核心样例库：60%（覆盖高频真实任务）
- 对抗样例库：25%（越权、提示注入、长链路干扰）
- 回归样例库：15%（历史事故与线上坏例）

11.2 排障优先级：先判定是评估问题还是策略问题

很多团队排障会直接改 prompt 或模型参数，但课程强调评估的重要性后，应先判断“是否因为评估器偏差导致假失败/假成功”。若评估器本身不稳，后续所有优化都可能偏航。

排障反模式

1. 不复现实验环境，直接根据线上截图改策略。
2. 只看平均分，不看失败分布和失败类型迁移。
3. 发现退化后立即回滚全部改动，导致无法定位根因。

11.3 标准化排障流程

步骤	执行动作	产出
定位	在固定评估集复现实验差异	失败样例清单 + 指标对比
归因	区分评估漂移/数据偏差/策略回归	根因标签与证据链
修复	最小改动验证假设	修复候选版本与风险说明
验证	全量回归 + 对抗测试 + 成本复核	上线建议与回滚条件

表 5: 可直接执行的 Agent 排障 workflow

11.4 长表：常见故障与处理策略

故障类型	症状	处理建议
工具误调用	参数格式正确但业务语义错误	在评估中加入语义约束检查；增加工具前置确认步骤
上下文漂移	多轮后目标偏离原任务	引入阶段性目标重述机制；加入计划一致性奖励
重试风暴	失败后连续无效重试导致成本飙升	设置失败路由与重试上限；按错误类型分流处理
评估振荡	小改动导致指标大幅波动	扩大评估样本；增加置信区间与分桶统计
开集退化	闭集提升但开集体验变差	增加 open-ended 评审权重；补充真实任务回放
权限越界	触发未经授权的写操作	强化动作白名单；高风险工具强制人审
记忆污染	错误历史进入长期记忆	增加记忆有效期与可信度标签；支持记忆回收
延迟失控	复杂任务响应时间不可接受	采用阶段式答复；并行化可独立子任务
解释失真	给出看似合理但与行为不符的理由	将解释与实际执行日志绑定校验
环境脆弱性	外部 API 抖动即导致任务失败	加入容错层、缓存层和降级策略

11.5 验收前检查清单

上线前 10 项必检

1. 评估集是否版本锁定并可复现。
2. 是否同时通过闭集与开集基线。
3. 高风险工具是否有权限门控。
4. 失败恢复路径是否经过压测。
5. 推理成本是否在预算区间。
6. 异常日志是否可追踪到具体动作。
7. 是否定义了上线后回滚阈值。
8. 关键指标是否有实时告警。
9. 人工接管流程是否可用。
10. 线上样例是否可回灌训练与评估。

11.6 本章小结

把评估与排障流程文档化，能够显著减少“经验主义调参”。课程强调的评估优先原则，在工程上可以直接落实为样例库、排障模板和上线检查清单。

12 总结与延伸

12.1 全课总结表

主题	课程核心判断	实践动作
训练流程	SFT → Reasoning RL → Agent RL	按阶段构建数据和奖励，避免一步到位
评估体系	evaluation is the key	闭集提速，开集保真，双轨并行
成本约束	训练成本与推理成本必须分开管理	建立端到端成本仪表盘
工具训练	synthetic tools 可扩展能力边界	合成数据与真实回放联合训练
系统瓶颈	长时任务常卡在内存与通信	异步调度、缓存与分层存储优化
安全上线	风险控制贯穿训练到运行时	分阶段放权与失败恢复评估

表 7: Lecture 11 的可执行结论

12.2 关键结论回顾

六条必须记住的结论

1. Agent 成功率取决于系统协同，而非单点模型分数。
2. Reasoning RL 是连接文本能力与执行能力的关键桥梁。
3. 数据与评估必须共设计，否则会互相污染。
4. 成本分析必须覆盖训练、评估和推理全链路。
5. 工具使用训练要关注决策质量，而不只是调用格式。
6. 开集任务里，失败恢复能力比一次成功更重要。

12.3 进一步阅读

- Berkeley RDI Agentic AI MOOC (Fall 2025) 课程主页与讲义。
- Yann Dubois 关于 LLM 训练与评估的公开分享与论文。
- ReAct、SWE-bench、WebArena、OSWorld 等 Agent 任务与评测工作。
- 工具使用与长任务强化学习相关实践 (tool simulation、async rollout)。
- 推理模型与 Agent 模型结合的后训练研究方向。